CREO — System Architecture

Senior Architect Design · Production-Grade · Pragmatic for Team Size & Timeline

v1.0 · Confidential

Greenfield · AWS · Modular Monolith → Microservices

⚠ MISSING INPUTS & ASSUMPTIONS (stated before proposing design)

Scale (not provided):	Assumed 10K creators + 250 enterprise orgs in 2 yrs; ~50M emails/mo; 50K peak send-events/min; ~500K subscriber records/tenant.
Team (not provided):	Assumed 8–12 engineers; strong TypeScript/React; some Go; AWS experience. → Favor managed services + monolith-first.
Budget (flexible):	Start lean (~$4–6K/mo), scale with revenue; avoid premature Kubernetes/multi-region.
Timeline (not provided):	MVP in 6 months, GA in 12 months; 2-year horizon for 10x scale.
Compliance:	SOC 2 Type II, ISO 27001, GDPR, CCPA (stated in product doc). HIPAA deferred to Phase 3.
Availability/latency:	99.95% API; 99.99% send pipeline; p95 < 300ms reads; send queue p99 < 5 min.
Existing infra:	Greenfield. AWS selected for breadth of managed services + team familiarity.
Data volume (not provided):	Assumed ~2 TB at launch, 4x/yr → ~32 TB by Y2. Event data dominates.
Read/write ratio:	~90% reads on CRMOS/content; send pipeline is write-heavy with bursty fan-out.
AI/LLM:	External LLM APIs (OpenAI/Anthropic) behind abstraction; no self-hosted GPU at launch.
Multi-tenancy:	Logical isolation via tenant_id + row-level security; dedicated schemas for enterprise tier.
Region:	Single region (us-east-1) at launch; EU region added at 10x for data residency.
ESP strategy:	Buy sending infrastructure (SES primary, SendGrid/Postmark failover) — build orchestration.
Payments:	Stripe for subscriptions/products/payouts; abstraction for future multi-processor.

1. HIGH-LEVEL ARCHITECTURE

Layered: Clients → Edge → API → Services → Async → Data → External Integrations

CLIENT LAYER

React / Next.js Web App
Creator Studio · Analytics · CRMOS UI
SSR + ISR · TanStack Query · Zustand

Mobile (Future / PWA)
Read-first companion app
React Native (deferred to Phase 2)

Embeddable Widgets
Signup forms · Preference center · Web
Preact · iframe-isolated · CSP-locked

Partner / API Consumers
REST / GraphQL / Webhooks / MCP
API keys · OAuth2 · rate-limited

EDGE LAYER

Cloudflare CDN + WAF + DDoS
Static assets · TLS termination · bot protection · rate limits
Edge caching · geo-routing · security headers

API Gateway (AWS API Gateway / Kong)
AuthN/Z · quotas · request routing · schema validation
JWT verification · mTLS · canary routing

Rate Limiter (Redis)
Per-tenant & per-API-key token buckets
Burst protection · abuse detection

API LAYER

GraphQL BFF (Apollo / GraphQL Yoga)
Frontend aggregation · auth context · DataLoader batching
Persisted queries · depth limiting · field-level auth

Public REST API + Webhooks + MCP Server
Versioned (/v1) · OpenAPI · idempotency keys · MCP for AI agents
Signed webhooks · retries with backoff · dead-letter queue

SERVICE LAYER — Modular Monolith (launch) → extractable microservices

Identity & Auth
OIDC · SAML · SCIM · sessions
RBAC enforcement point
Tech: Node.js + TypeScript, Keycloak-style
Interfaces: REST, OIDC, SCIM
Responsibilities: users, tenants, roles, API keys, MFA, session revocation
→ Stateless; scales horizontally

Newsletter Content
Editor backend · versioning · templates
Dynamic content · merge rendering
Tech: Node.js, MJML, Handlebars
Interfaces: gRPC internal, REST public
Responsibilities: drafts, approvals, scheduling, rendering to email HTML
→ Read-heavy; Postgres + Redis cache

Deliverability Engine
Send orchestration · suppression
Feedback loops · IP warming · isolation
Tech: Go (high-throughput workers)
Interfaces: gRPC + async queue
Responsibilities: batch fan-out, bounce classification, ESP failover, audit log
→ Write-heavy; separate deployment in Phase 2

CRMOS
Profiles · lifecycle · scoring · consent
Timeline · segmentation · team inbox
Tech: Node.js + TypeScript
Interfaces: gRPC + GraphQL subgraph
Responsibilities: unified profile, stage transitions, propensity, consent ledger
→ Read-heavy; Postgres + Redis

Automation Engine
Workflows · triggers · branching
Delays · webhooks · versioned deploy
Tech: Temporal (durable execution)
Interfaces: Temporal SDK, internal gRPC
Responsibilities: run workflows, retries, compensation, audit history
→ Stateful; Temporal cluster (managed)

Monetization
Subscriptions · products · ads · boosts
Payouts · revenue recognition
Tech: Node.js + Stripe SDK
Interfaces: REST + Stripe webhooks
Responsibilities: plans, invoices, entitlements, ad matching, payouts
→ PCI scope minimized via Stripe

Analytics
Semantic layer · cohorts · dashboards
Real-time + batch · embedded
Tech: Node.js + ClickHouse client
Interfaces: GraphQL subgraph
Responsibilities: query planning, materialized views, KPI definitions
→ Extract to own service in Phase 2

Governance / RBAC
Policy engine · data dictionary
Lifecycle · audit log · retention
Tech: OPA (Open Policy Agent) + Node
Interfaces: sidecar (policy decisions)
Responsibilities: attribute-based access, consent enforcement, immutable audit
→ Cross-cutting; every service calls it

Data Ingestion
Connectors · CDC · schema validation
Dedup · lineage at entry
Tech: Node.js + Kafka Connect + dbt
Interfaces: push API, pull API, CDC
Responsibilities: CRM/e-com/partner feeds, validation, dead-letter handling
→ Batch + streaming; isolated workers

Notification & Webhook
Outbound webhooks · in-app events
Retries · DLQ · signing
Tech: Node.js + SQS/SNS
Interfaces: HTTP webhooks, SNS
Responsibilities: deliver events to partners, sign payloads, track failures
→ Async; independently scalable

ASYNC BACKBONE

Event Bus — Apache Kafka (MSK) / AWS SQS+SNS for lower-throughput paths
Topics: send.requested · email.delivered · email.bounced · subscriber.created · consent.changed · payment.succeeded · workflow.triggered

DATA LAYER

PostgreSQL (Aurora)
OLTP — Source of truth
tenants · users · publications · subscriber_profiles · consents · audit_log · workflows
Multi-AZ · read replicas · RLS

ClickHouse
OLAP — Analytics & events
sends · opens · clicks · bounces · engagement · lifecycle events · deliverability time-series
Partitioned · TTL · cluster in Phase 2

Redis (ElastiCache)
Cache · rate limiting · sessions
session tokens · rate-limit buckets · hot subscriber lookups · distributed locks · Temporal visibility cache
Cluster mode at 10x

Amazon S3
Object storage / data lake
images · exports · email archives · raw ingestion landing · backups · Iceberg tables for analytics
Lifecycle policies · versioning

OpenSearch
Full-text search
subscriber search · content search · audit log search · segment preview support ticket search
Index-per-tenant option

pgvector / Pinecone
Vector DB for AI
embeddings for semantic search · AI agent memory · content similarity · subscriber look-alike modeling
pgvector at launch → Pinecone at 10x

EXTERNAL INTEGRATIONS (via Integration Fabric)

ESP Providers
SES (primary) · SendGrid / Postmark
failover · provider-agnostic abstraction
per-tenant reputation isolation

Stripe
Payments · subscriptions · payouts
webhooks · Connect for creator payouts
tax · revenue recognition events

CRM Systems
Salesforce · HubSpot · Dynamics
bidirectional sync via integration service
field mapping · conflict resolution

LLM Providers
OpenAI · Anthropic (abstracted)
prompt versioning · cost controls
PII redaction before inference

Email Validation
ZeroBounce · Kickbox · internal
bot detection · disposable blocking
fallback chain · cached results

Observability
OpenTelemetry → Grafana/Datadog
traces · metrics · logs · Sentry
SLO monitoring · PagerDuty

CROSS-CUTTING PLANES (apply to every layer)

Governance Plane: OPA policy sidecar · RBAC · audit log (append-only) · data catalog · retention & deletion
Security Plane: WAF · mTLS between services · KMS encryption at rest · secrets in AWS Secrets Manager · DLP on exports
Observability Plane: OTel tracing (correlation IDs) · RED/USE metrics · structured logs · SLOs + error budgets

2. KEY DATA FLOWS

Flow 1 — Subscriber Signup (sync write + async fan-out)

① Widget/Form → Edge (WAF, rate limit) → API Gateway → GraphQL BFF
② Sync: Identity validates → Content Svc checks publication → CRMOS creates profile + consent record → Postgres (transactional)
③ Sync: Email validation (internal + ZeroBounce) → bot/disposable check → suppression check → respond 201
④ Async: emit `subscriber.created` → Kafka → Analytics (ClickHouse) · Automation Engine (welcome workflow) · Notification (webhook to partner)

Flow 2 — Newsletter Send (async pipeline, highest volume)

① Scheduler (Temporal) fires at send time → Deliverability Engine builds send job → resolves segment (CRMOS) → renders (Content Svc)
② Suppression check (global + publication + transactional) → filters recipients → splits into batches → publishes to `send.requested`
③ Go workers consume → call ESP (SES primary, failover to SendGrid/Postmark) → capture provider message ID → log to ClickHouse
④ Feedback loop: ESP webhooks → Deliverability Engine → classify (delivered/bounced/complained) → update suppression → emit `email.delivered` → CRMOS timeline + Analytics

Flow 3 — Analytics Query (sync read, p95 < 300ms)

① Dashboard → GraphQL BFF → Analytics Service → checks Redis cache for query hash
② Cache miss: Analytics Svc compiles semantic query → ClickHouse (materialized views for common cohorts) → returns aggregated rows
③ Result cached in Redis (TTL by query type: 60s for real-time, 1h for cohort) → returned to BFF → formatted for UI
④ Governance: every query passes OPA policy (tenant isolation + role-based field filtering) → query + result logged to audit

3. SCALABILITY PATH

Phase 1 — Launch (0–6 months)

Target: 10K creators · 50M emails/mo

- Modular monolith on ECS Fargate (Node.js)
- Aurora Postgres (Multi-AZ) + ElastiCache Redis
- MSK Kafka (small) + SQS for webhooks
- ClickHouse Cloud (single node)
- CloudFront + WAF; single region (us-east-1)
- Temporal Cloud (managed)

Trigger to advance: p95 API > 400ms sustained or send queue p99 > 10 min, or DB CPU > 70%

Est. cost: $4–6K/mo

Phase 2 — ~10x (12–18 months)

Target: 100K creators · 500M emails/mo

- Extract Deliverability + Analytics into separate services
- ClickHouse cluster (3+ nodes, replicated)
- Postgres read replicas; partition largest tables
- Redis Cluster; Kafka topic partitioning by tenant
- Add EU region for data residency (EU enterprise)
- pgvector → Pinecone for semantic search at scale

Trigger to advance: 10K+ req/s peak, multi-region latency SLA pressure, or one service's deploy cadence blocks others

Est. cost: $15–30K/mo

Phase 3 — 2-Year Target

Target: 250K creators · 2B emails/mo

- Full microservices on EKS (Kubernetes)
- Cell-based architecture (per-region cells)
- Multi-region active-active (US + EU + APAC)
- Global CDN; regional data planes
- Dedicated tenancy option for enterprise
- Custom MTA evaluation at >500M emails/mo

Steady state: horizontal scaling within cell; new region = new cell (no core rewrite)

Est. cost: $60–120K/mo

4. KEY TRADEOFFS

Decision	Chosen Option	Sacrifices	Reverse When…
Monolith vs. Microservices Modular monolith first	Single deployable, shared DB, clear module boundaries	Independent scaling & deploy per module; team autonomy limited at first	Team > 15 engineers OR one module's load starves others OR deploy cadence blocks teams
SQL vs. NoSQL Postgres + ClickHouse	Postgres OLTP + ClickHouse OLAP (no DynamoDB/Mongo)	Horizontal write scale on Postgres requires sharding/citus at extreme scale	Single table > 5TB OR write throughput bottleneck OR multi-region active-active
Build vs. Buy — ESP Buy sending infra	SES primary, SendGrid/Postmark failover; build orchestration layer	No direct MTA control; per-ESP limits; negotiated pricing needed at scale	> 500M emails/mo OR margin pressure justifies evaluating custom MTA / dedicated IPs
Workflow engine Temporal (managed)	Durable execution, retries, compensation, audit history	Operational complexity; vendor lock-in; cost at high workflow volume	< 10 workflow types OR team lacks Temporal ops capacity → SQS + Lambda
Region strategy Single region at launch	us-east-1; EU region added in Phase 2 for data residency	Higher latency for far users; DR is region-pair, not active-active	EU enterprise deals require residency OR p95 latency > 300ms for APAC/EU users

DESIGN PRINCIPLES APPLIED

- **Simplicity first:** modular monolith + managed services; extract only when load or team forces it.
- **Governance is a cross-cutting plane, not a feature:** OPA sidecar + audit log applied at every service boundary.
- **Async by default for fan-out:** send pipeline, webhooks, analytics ingestion all decoupled via Kafka/SQS.
- **Buy infrastructure, build orchestration:** ESPs, Stripe, LLMs, Temporal are bought; the moat is the governed relationship layer.
- **Tenant isolation everywhere:** tenant_id in every row, RLS, per-tenant rate limits, optional dedicated schemas.

Legend: Blue = clients · Indigo = edge · Purple = API · Green = services
Amber = async · Teal = data · Gray = external · Red dashed = cross-cutting
Yellow = assumptions · Green/Blue/Purple = scalability phases